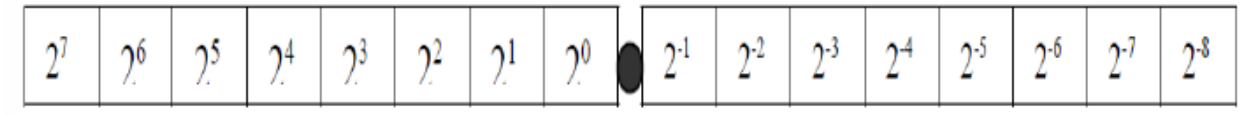
Electrical and Computer Engineering
ECE3622 Embedded Systems Design
Dennis Silage, PhD
silage@temple.edu

Fixed Point Arithmetic

Numbers can be represented for calculation in a processor as either fixed point or floating-point. A fixed-point representation maintains the decimal point as a fixed position which facilities arithmetic operations. However, a fixed-point representation requires a large number of bits to represent larger numbers or a reasonably accurate result with fractional numbers.

A fixed point number consists of two parts: the integer and fractional parts. Floating point representation allows the decimal point to *float* to different locations within the number depending upon the magnitude. A floating point number also consists of two parts: the exponent and the mantissa.

Although the FPGA can support both fixed and floating point numbers, most applications utilize fixed point number systems as they are simpler to implement than floating point ones.



Fixed point number system

The example fixed-point number system above is capable of representing an unsigned number of 0.0 to 255.9906375. If the most significant bit (2^7) is now the sign bit the example fixed point number system can represent a signed number of –128.9906375 to 127.9906375 using twos complement representation.

A typical design can use either unsigned or signed fixed-point numbers. This design choice is determined by the arithmetic algorithm being implemented. Unsigned fixed-point numbers are capable of representing a positive integer in the range of 0 to 2^{n-1} with some fractional part. Signed fixed-point numbers use the twos complement number system to represent both positive and negative integers in the range $-(2^{n-1})$ to $+(2^{n-1} - 1)$ with some factional part.

The normal way of representing the split between integer, fractional bits within a fixed-point number is x,y where x represents the number of integer bits and y the number of

fractional bits. For example 8,8 represents 8 integer bits and 8 fractional bits while 16,0 represents 16 integer and 0 fractional.

Usually the choice of the number of integer and fractional bits required is determined at design time normally following conversion from a floating point algorithm. The Mathworks *Fixed Point Toolbox* has been used in the SCDL (*www.temple.edu/scdl*) to perform this task.

Because of the inherent flexibility of an FPGA a fixed-point number can be represented by any bit length. The number of integer bits required depends upon the maximum integer value the fixed-point number is required to process, while the number of fractional bits will depend upon the desired accuracy of the final result.

To determine the number of integer bits required we can use the following equation:

$$\text{Integer Bits} = \mathit{Ceil}\ (\mathit{log}_{10}\ \text{MAXVALUE}/\mathit{log}_{10} 2)$$

Where *Ceil* returns the smallest integer great than or equal to a given number and MAXVALUE is the maximum expected value of the number to be represented. For example for a number between 0.0 and 423.0 9 integer bits would be required.

The number of fractional bits is determined by the acceptable resolution. For example, a single decimal fractional resolution, as in N.0 to N.9, with ±1% accuracy requires at least eight fractional bits.

For addition, subtraction or division the decimal points of both fixed-point numbers must be aligned. An x,8 number can only added to, subtracted from or divided by a number which is also in a x,8 representation.

To perform arithmetic operations with fixed-point numbers of different x,y format we must first ensure the decimal points are aligned. Although it is not strictly necessary to align the decimal points of fixed-point numbers for division, the operation must be performed carefully to ensure the result is scaled correctly and not inadvertently negative. For division by a fixed constant its reciprocal can be calculated and then the operation multiplies the fixed-point number by the reciprocal for efficiency.

When multiplying two numbers together the decimal points do not need to be aligned as the multiplication will provide a result which is X1 + X2, Y1 + Y2 wide. Multiplying two fixed-point numbers formatted 14,2 and 10,6 produces a result formatted as 24 integer bits and 8 fractional bits.

As an example of the analysis, a positive input signal x between 0 and 10 with 4 decimal bits and 12 fractional bits store is stored in a 16 bit input vector. The input signal x is processed by a non-linear equation to produce an output y:

$$y = -0.0088\, x^2 + 1.7673\, x + 131.29$$

The processing equation has three constants A, B and C which require scaling to implement as fixed point numbers. An FPGA means can use different scaling for each constant to optimize the performance.

	Unscaled	Decimal Point Location	Resultant Format	Scaled
A	-0.0088	16	0,16	-577
B	1.7673	15	1,15	57910
C	131.29	8	8,8	131

The processing equation has three constants A, B and C which require scaling to implement as fixed point numbers. An FPGA means can use different scaling for each constant to optimize the performance.

The scaling for Ax^2 and Bx can be analyzed as follows:

x format: 4,12 x^2 format: 8,24 Ax^2 format: 8,40
x format: 4,12 Bx format: 5,27

The addition of constant C requires the alignment of the fixed-point decimal point. Ax^2 and Bx are scaled to C. Ax^2 is scaled by 2^{32} and Bx is scaled by 2^{19} resulting in an 8,8 format which is aligned to the constant C.